AMLS

Theodor Hagen

July 2026

1 Introduction

2 Dataset Exploration and Cleaning

This section examines the composition and technical properties of the training data before modeling. The analysis focuses on class imbalance, potentially label-revealing image properties, and the deterministic cleaning decisions used to obtain a reliable input dataset for the later modeling stages.

2.1 Dataset Exploration

The provided dataset consists of encoded images and a source label `source_class`. Class 0 represents real images, whereas classes 1–5 represent images generated by five different generative models. For the binary classification task, the five generated-image classes are merged into a single AI-generated class.

Although the six original source classes are approximately balanced, merging five of them produces a substantially imbalanced binary problem. As shown in Figure 1, the resulting ratio of real to AI-generated images is approximately 1:5. Consequently, a classifier that always predicts the majority class would achieve a high accuracy despite being unable to identify real images. Accuracy alone is therefore not an appropriate evaluation metric for this task. The later experiments instead focus on the recall of the AI-generated class while explicitly controlling the false-positive rate on real images.

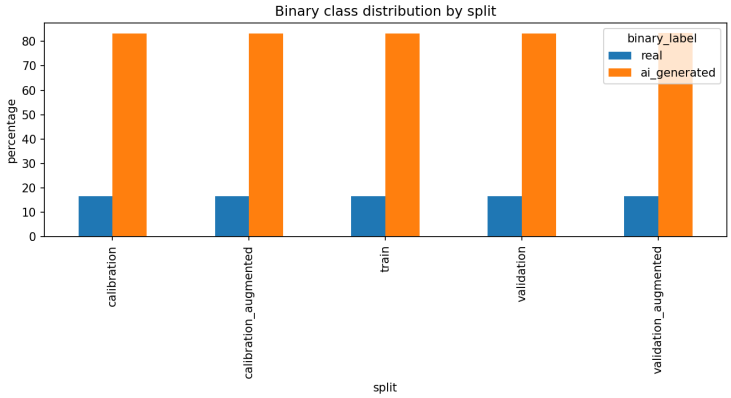


Figure 1: Binary class distribution after merging source classes 1–5 into the AI-generated class. The percentages reveal an imbalance of approximately five AI-generated images per real image.

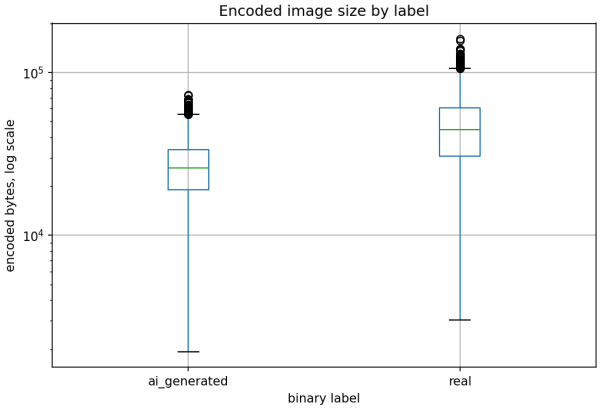
Figure 2a shows the distribution of the encoded image size. Real images have a higher typical encoded size than AI-generated images. Although the distributions overlap, their different central tendencies indicate that encoded size contains information about the target class. Since encoded size depends on factors such as image resolution, compression quality, file format, and image complexity, it should not be interpreted as an intrinsic indicator of whether an image is generated. Instead, it represents a potential dataset shortcut.

A similar issue is visible in the aspect-ratio distribution in Figure 2b. The data contain several pronounced peaks corresponding to frequently occurring image dimensions, and their relative frequencies differ between real and AI-generated images. This suggests that certain image shapes are more common in one class than in the other and could therefore provide additional non-semantic information about the label.

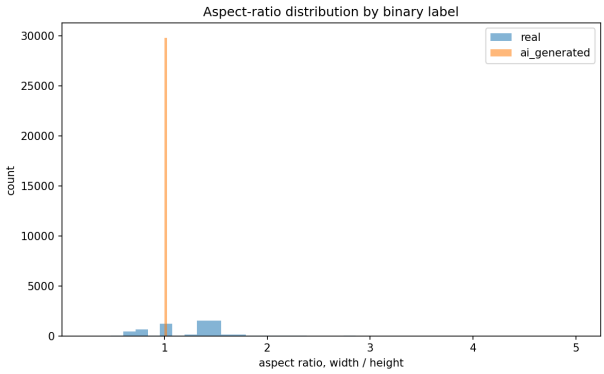
The resolution distribution reveals an even stronger dataset-specific pattern. As shown in Figure 2c, AI-generated images are concentrated at only two distinct megapixel values, whereas real images cover a substantially wider range of resolutions. Figure 2d confirms that these two clusters correspond to image sizes of approximately 270×270 and 320×320 pixels. In contrast, the real images occur at many different combinations of width and

height. Original image resolution could therefore serve as a strong shortcut for separating the classes without analyzing their visual content.

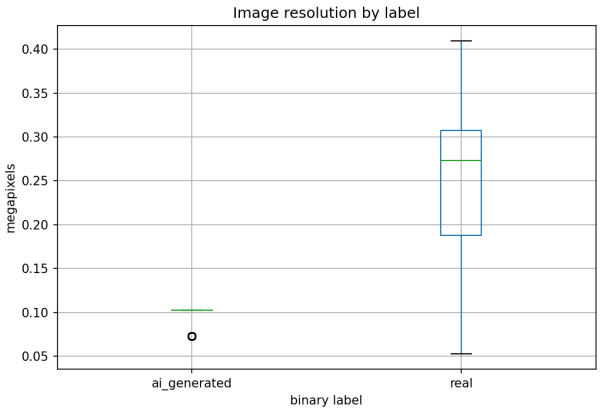
Taken together, these observations indicate that a classifier could associate low-level properties such as encoded file size, aspect ratio, or original resolution with the target class. To reduce this risk, all images were converted to a common input size during model preparation, and metadata such as encoded file size, original dimensions, and file format was not provided to the classifier.



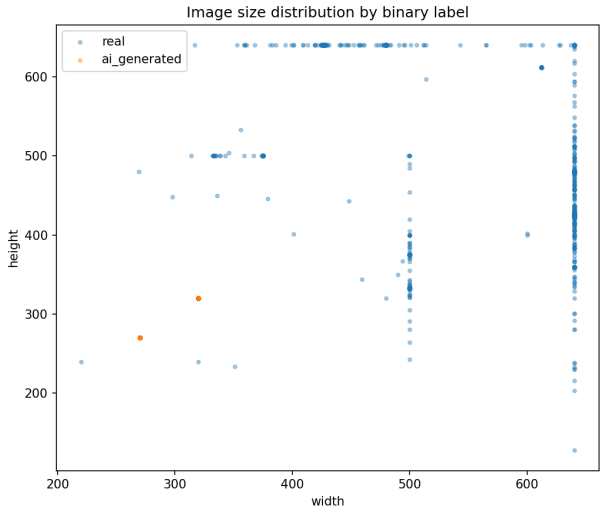
(a) Encoded image-size distribution on a logarithmic scale.



(b) Aspect-ratio distribution separated by binary label.



(c) Megapixel distribution by binary label. AI-generated images are concentrated at two discrete resolution levels, while real images exhibit a much broader distribution.



(d) Image width and height by binary label. The two AI-generated clusters correspond to approximately 270×270 and 320×320 pixels, whereas real images occur at many different resolutions.

Figure 2: Image properties that may act as shortcuts for predicting the target class. All four properties differ systematically between real and AI-generated images, although none of them should be treated as semantic evidence of image generation.

2.2 Deterministic Cleaning Pipeline

The cleaning pipeline is implemented in `clean.py`. It checks whether each encoded image can be decoded correctly and removes samples with invalid image data or dimensions. For every valid image, the script records basic metadata such as width, height, aspect ratio, file size, format, and color mode. The original source label is retained, while source classes 1–5 are additionally merged into the binary AI-generated class.

The original parquet files are not modified. Cleaned sample information, summary statistics, and plots are stored to `artifacts/task01/`.

3 Modeling and Calibration

This section describes the CPU-friendly classification pipeline developed for Task 1.2. It covers data preparation, the comparison of classical and neural models, training, automatic threshold calibration, and the independent evaluation of the selected model.

3.1 Methodology

Preprocessing. Before model training, all labeled dataset splits were processed using the same deterministic preprocessing pipeline. Each image was decoded, corrected according to its EXIF orientation, converted to RGB, and resized to 128×128 pixels while preserving its original aspect ratio. A centered crop was applied after resizing to avoid geometric distortions caused by directly stretching non-square images. The processed images were stored as channel-first `uint8` NumPy arrays and converted to floating-point tensors only during training, reducing storage requirements and avoiding repeated image decoding across epochs.

Channel-wise mean and standard deviation values were computed exclusively from the training split and used to normalize all dataset splits. No random transformations were applied during this stage, since augmentation is considered separately in Task 1.3. The same preprocessing function is reused during inference to ensure consistency between training and prediction.

Classical models. Two classical models were trained as computationally inexpensive baselines: logistic regression and gradient boosting. Both models operate on 34 deterministic image-level features extracted from the prepared images. These features describe color-channel statistics, grayscale quantiles and entropy, saturation, gradient and Laplacian responses, edge frequency, channel correlations, blockiness, and differences between the image center and border.

Original image dimensions, encoded file size, and file format were deliberately excluded because the exploratory analysis identified these properties as potential dataset shortcuts. Class weighting was used for logistic regression to account for the imbalanced binary class distribution. Gradient boosting was included because it can represent nonlinear interactions between the engineered features that cannot be captured by a linear classifier.

CNN architecture. The neural model is a compact CNN trained directly on the prepared image pixels. It contains six 3×3 convolutional blocks with channel widths $16, 16, 32, 32, 64, 64$. Each convolution is followed by batch normalization and a ReLU activation. The first block at each channel width uses a stride of two, reducing the spatial resolution from 128×128 to 64×64, 32×32, and finally 16×16.

This early spatial downsampling substantially reduces the computational cost of the later convolutional layers and makes training feasible under the CPU time constraint. Adaptive average pooling converts the final feature maps into a 64-dimensional representation. Dropout with probability 0.2 is applied before a linear two-class output layer. The complete CNN contains 72,434 trainable parameters and is trained from scratch without pretrained weights.

Training and checkpoint selection. A deterministic stratified split reserves 10% of the training data as an internal monitoring set. This leaves 26,719 samples for parameter optimization and 2,969 samples for checkpoint selection. The official calibration and validation splits are not used for gradient-based training.

Inverse-frequency class weights are applied in the cross-entropy loss to reduce the effect of the approximately 1:5 class imbalance. Training uses AdamW with a maximum learning rate of 3×10^{-3}, weight decay of 10^{-4}, a batch size of 256, and a OneCycle learning-rate schedule. Training is limited to 15 epochs and uses an early-stopping patience of five epochs. A fixed random seed of 42 and a limit of eight CPU threads are used to make the experiment reproducible.

After each epoch, a temporary operating threshold is selected on the internal monitoring set. Checkpoints are ranked primarily by AI recall while limiting the false-positive rate on real images to 17%. ROC–AUC and monitoring loss are used as secondary criteria. The stricter 17% target leaves a safety margin for the required 20% limit on the independent validation split.

Threshold calibration. The operating threshold of each model is determined automatically using the official calibration split. Among all thresholds satisfying the specified false-positive constraint, the threshold with the highest AI recall is selected. The classical models use a calibration target of 18%, while the final CNN uses the stricter target of 17%.

For the CNN, this procedure produced a threshold of 0.6990, corresponding to a calibration false-positive rate of 16.77% and an AI recall of 82.33%. After calibration, the checkpoint and threshold are fixed and applied without modification to the original and augmented validation splits.

3.2 Evaluation

Table 1 compares the three investigated model families on the original validation split. The gradient boosting model clearly outperforms logistic regression, increasing AI recall from 55.9% to 66.7%. This indicates that nonlinear relationships between the engineered features contain useful information. However, neither classical model reaches the target recall of 0.8.

The CNN achieves substantially better performance than both feature-based models. It reaches an AI recall of 81.8%, while maintaining a real-image false-positive rate below the required 20%. It is therefore selected as the final Task 1.2 model.

Model	FPR real	Recall AI	Precision AI	ROC–AUC	PR–AUC
Logistic regression	18.1%	55.9%	93.9%	0.773	0.927
Gradient boosting	19.1%	66.7%	94.5%	0.833	0.955
CNN	**18.6%**	**81.8%**	**95.6%**	**0.890**	**0.973**

Table 1: Performance on the original validation split. The operating threshold of each model was selected independently using the calibration split.

On the original validation split, the CNN produces 153 true negatives, 35 false positives, 170 false negatives, and 766 true positives. This corresponds to an AI recall of 81.84%, an AI precision of 95.63%, and a real-image false-positive rate of 18.62%. The model therefore exceeds the target recall of 0.8 while respecting the required false-positive limit. Its threshold-independent ROC–AUC and PR–AUC values are 0.890 and 0.973, respectively.

The same checkpoint and threshold were also evaluated on `validation_augmented`. AI recall decreases to 72.89%, while the false-positive rate increases to 34.76%. AI precision remains relatively high at 91.31%, but ROC–AUC decreases to 0.783. The degradation shows that the CNN learns useful visual information but is not sufficiently robust to the image modifications represented by the augmented split. In particular, the increased false-positive rate indicates that modified real images are frequently classified as AI-generated. This result motivates the augmentation strategy developed in Task 1.3.

The complete training and evaluation procedure required 735.9 seconds. The provided reference CNN required 173.142 seconds on the same machine, resulting in a runtime ratio of approximately 4.25. The final pipeline therefore remains within the permitted five-times-reference training budget.

During inference, the saved checkpoint, training-set normalization statistics, and calibrated threshold are loaded. Prediction images undergo the same deterministic preprocessing as the training data, and the resulting binary labels are written to `artifacts/task02/predictions.csv`.

4 Data Augmentation and Robustness

This section extends the CNN from Task 1.2 with an augmentation-based fine-tuning strategy. The objective is to improve its robustness to common image modifications while maintaining the required false-positive rate on both the original and augmented data. The resulting model is compared with the Task 1.2 CNN using the same evaluation protocol.

4.1 Methodology Augmented

The Task 1.3 model retains the CNN architecture and deterministic preprocessing pipeline introduced in Task 1.2. Instead of training a new model from randomly initialized weights, fine-tuning starts from the best Task 1.2 checkpoint. This allows the model to preserve the visual features learned from the original training data while adapting them to modified images.

Robustness transformations are applied online to each training batch after normalization has temporarily been reversed. The pipeline contains four independently sampled operations. With a probability of 0.35, an image is downscaled by a factor selected from 0.5, 0.65, and 0.8, and then resized back to 128×128 pixels using bilinear interpolation. This simulates information loss caused by repeated resizing. A 3×3 average blur is applied with probability 0.25. Compression-like artifacts are approximated with probability 0.35 by quantizing the pixel intensities to 32, 64, or 128 discrete levels. Finally, Gaussian noise with a standard deviation selected from 0.005, 0.01, and 0.02 is added with probability 0.15.

Because the transformations are sampled independently, several operations may be combined for the same image. At the same time, approximately 27% of the training samples remain unchanged. This mixture exposes the model to modified images without completely discarding its performance on clean data. The operations were kept lightweight so that they could be executed directly on CPU tensors without storing additional augmented image copies.

The same stratified training and monitoring split as in Task 1.2 is used. Training continues with the class-weighted cross-entropy loss, AdamW, a batch size of 256, and a weight decay of 10^{-4}. Since the model is fine-tuned rather than trained from scratch, the maximum learning rate is reduced to 8×10^{-4}. A OneCycle learning-rate schedule is used over a maximum of ten epochs, with an early-stopping patience of three epochs.

For checkpoint selection, the internal monitoring images are evaluated both without transformations and with a fixed augmentation sequence. A conservative threshold is chosen that limits the false-positive rate to 19% for both versions of the monitoring data. Checkpoints are ranked primarily according to the lower of their clean and augmented AI recall. Mean recall and mean ROC–AUC are used as secondary criteria. This procedure discourages selecting a checkpoint that performs well on one version of the data but poorly on the other. The best checkpoint was obtained after epoch 8.

After checkpoint selection, the final operating threshold is determined using both `calibration` and `calibration_augmented`. A separate recall-maximizing threshold under a 19% false-positive limit is calculated for each calibration split. The larger of the two thresholds is then selected, ensuring that the same final operating point satisfies the calibration constraint on both clean and augmented images.

This procedure produces a final threshold of 0.6309. On the original calibration split, it results in an AI recall of 80.90% and an FPR of 16.77%. On `calibration_augmented`, the corresponding recall is 62.51%, while the FPR remains at 18.69%.

4.2 Evaluation

Table 2 compares the Task 1.2 CNN with the augmented CNN on both validation splits. Each model is evaluated using its own fixed calibration threshold.

Model	Split	FPR real	Recall AI	Precision AI	ROC–AUC	PR–AUC
Task 1.2 CNN	Original	18.6%	81.8%	95.6%	0.890	0.973
Task 1.2 CNN	Augmented	34.8%	72.9%	91.3%	0.783	0.948
Augmented CNN	Original	18.1%	79.0%	95.6%	0.887	0.973
Augmented CNN	**Augmented**	**19.8%**	**61.3%**	**93.9%**	**0.786**	**0.949**

Table 2: Comparison of the Task 1.2 CNN and the augmented CNN on the original and augmented validation splits.

On the original validation split, the augmented CNN achieves an AI recall of 78.95% and an FPR of 18.09%. Compared with the Task 1.2 CNN, recall decreases by approximately 2.9 percentage points, while the FPR is reduced by approximately 0.5 percentage points. The model therefore retains competitive performance on unmodified images and continues to satisfy the required false-positive constraint.

The main difference is visible on `validation_augmented`. The Task 1.2 CNN obtains a higher recall of 72.89%, but its FPR increases to 34.76%, substantially exceeding the allowed 20%. Its operating point is therefore not valid for the augmented data. In contrast, the augmented CNN reduces the FPR to 19.79% and reaches an AI recall of 61.26%. It consequently satisfies both the false-positive constraint and the target augmented recall of at least 0.6.

This result involves a clear trade-off. The augmented model does not improve recall independently of the operating constraint; instead, it sacrifices some recall to avoid incorrectly accusing a large proportion of modified real images. Its threshold-independent performance changes only slightly: ROC–AUC on the augmented validation data increases from 0.783 to 0.786, while PR–AUC increases from 0.9482 to 0.9486. The main improvement therefore comes from combining augmentation-based fine-tuning with a threshold that is calibrated conservatively across both data variants.

The augmentation training itself required 639.7 seconds, while the complete training, calibration, and evaluation script required 696.5 seconds. This corresponds to approximately 4.02 times the reference runtime of 173.142 seconds and therefore remains within the permitted five-times runtime budget.

During inference, no random augmentation is applied. The saved CNN, training-set normalization statistics, and calibrated threshold are loaded, and prediction images undergo the same deterministic preprocessing used in Task 1.2. The resulting labels are written to `artifacts/task03/predictions.csv`.

5 Explainability and Failure Analysis

This section analyzes the behavior of the final augmented CNN using signed class activation maps and occlusion sensitivity. The objective is to identify which image regions support predictions of the real and AI-generated classes, to examine representative correct and incorrect decisions, and to assess the limitations of the resulting explanations.

5.1 Methodology Explainability

The analysis uses the final Task 1.3 checkpoint and its calibrated decision threshold of 0.6309. Predictions on the original validation split are divided into true positives, true negatives, false positives, and false negatives. One confident example is selected from each category. In each case, the selected image has the largest absolute distance between its predicted AI probability and the decision threshold. The resulting figure therefore contains one representative example of each type of correct and incorrect classification without relying on manual selection.

The primary explanation method is a signed class activation map. Let $A_k(i, j)$ denote the activation at spatial position (i, j) in the k-th feature map of the final convolutional representation. The signed map is computed as

$$M(i, j) = \sum_k \left(w_k^{\mathrm{AI}} - w_k^{\mathrm{Real}}\right) A_k(i, j),$$

where w_k^{AI} and w_k^{Real} are the weights connecting feature map k to the two output logits. Positive values increase the AI logit relative to the real logit, while negative values support the real class. Red regions in the visualizations therefore represent evidence for an AI-generated prediction, whereas blue regions represent evidence for a real image.

Occlusion sensitivity is used as an additional plausibility check. For each selected image, a 24×24 patch is replaced by the channel-wise training mean. The patch is moved across the 128×128 input with a stride of 12, producing 100 modified variants of each image. The change in AI probability is measured for every patch position.

A positive occlusion value means that removing the corresponding region decreases the AI probability, indicating that the region supported the AI class. A negative value means that removal increases the AI probability, indicating that the original region supported the real class. Pearson correlation between the resized signed CAM and the occlusion map is reported as a measure of their spatial agreement.

5.2 Qualitative Analysis

Figure 3 compares the signed CAM and occlusion sensitivity for one confident example from each prediction category. Every row contains the original image, the signed CAM, and the corresponding occlusion map.

The true-positive example is an AI-generated image dominated by repeated horizontal structures. Both positive and negative contributions are present, but the strongest positive evidence occurs around several of the regular horizontal transitions. This suggests that the CNN detects low-level structural patterns rather than identifying a particular semantic object as AI-generated.

The true-negative example is a real outdoor scene containing people and animals. Its signed CAM is predominantly blue across large parts of the image, indicating broadly distributed evidence for the real class. The decision does not appear to depend exclusively on one person or object, but instead on the overall combination of textures and scene structure.

The false-positive example contains an orange-and-white construction barrier. Strong positive contributions occur around the repeated stripes and the lower central structure. These regular, high-contrast patterns are interpreted as evidence for the AI class even though the image is real. This example indicates that geometric repetition and sharply separated color regions can act as misleading cues. Since this conclusion is based on an individual example, it should be treated as evidence of a possible failure mode rather than proof of a general rule.

The false-negative example is an AI-generated landscape with flowers and mountains. Large parts of the foreground receive negative contributions and therefore support the real class. Its coherent composition, natural color transitions, and smooth landscape textures appear sufficiently photographic to hide many of the cues learned for AI-generated images. This demonstrates that photorealistic generated scenes without obvious local artifacts remain difficult for the detector.

The Pearson correlations displayed in the figure indicate how closely the two explanation methods agree spatially for each example. Agreement between the methods increases confidence that the highlighted regions have a measurable influence on the prediction. However, perfect agreement is not expected because the two approaches describe different aspects of the model. Signed CAM decomposes the final convolutional representation, whereas occlusion sensitivity measures the effect of directly modifying the input image.

5.3 Generator-Specific Failure Analysis

The detector's performance also differs between the five AI source classes. Table 3 reports recall separately for each generator on the original and augmented validation splits.

On the original validation split, recall ranges from 72.7% for SD 3 to 88.3% for DALL-E 3. Performance decreases for every source class after image modification. The strongest augmented result is obtained for DALL-E 3 with a recall of 76.5%, while SD 3 and Midjourney are the most difficult sources, with recalls of 50.0% and 53.5%, respectively.

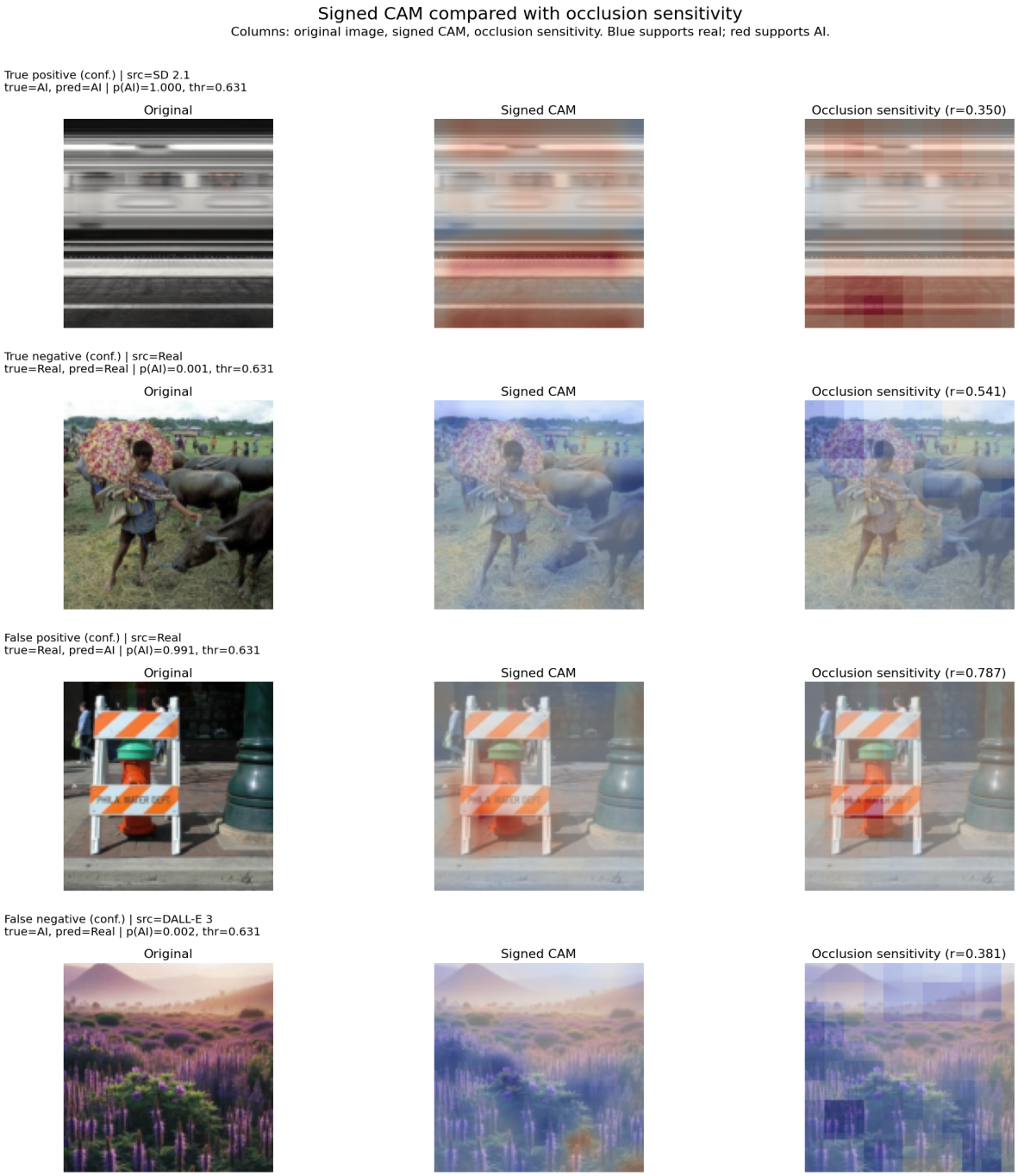


Figure 3: Comparison of signed class activation maps and occlusion sensitivity for one confident true positive, true negative, false positive, and false negative from the original validation split. Red regions support the AI-generated class relative to the real class, while blue regions support the real class. The value r denotes the Pearson correlation between the signed CAM and the occlusion map.

Source class	Original recall	Augmented recall
SD 2.1	75.4%	61.7%
SDXL	85.1%	64.7%
SD 3	72.7%	50.0%
DALL-E 3	88.3%	76.5%
Midjourney	73.1%	53.5%

Table 3: AI recall of the final augmented CNN for each original generator class on the original and augmented validation splits.

These differences indicate that the CNN has not learned a single universal property shared equally by all AI-generated images. Instead, some learned cues appear more strongly in particular generator families. Resizing, blurring, quantization, and noise can weaken these cues, causing additional false negatives on the augmented data.

5.4 Limitations and Conclusions

Although the signed CAM exactly decomposes the final logit difference, it does not provide a complete description of all computations performed by the CNN. The native CAM resolution is only 16×16, corresponding to the final convolutional feature maps. It is enlarged to 128×128 for visualization, which makes the resulting overlay appear more spatially precise than the underlying explanation actually is.

Occlusion sensitivity also has important limitations. Replacing an image patch with the channel-wise training mean creates an artificial input that may not occur naturally. A change in prediction can therefore be caused both by the removal of relevant information and by the introduction of an unusual region. Occlusion should consequently be interpreted as a complementary plausibility check rather than as ground-truth evidence that the CAM is correct.

The selected examples are deterministic, but four individual images cannot fully represent the behavior of the complete validation dataset. The generator-specific evaluation therefore complements the visual analysis with an aggregate view of the model's weaknesses.

Overall, the explanations suggest that the CNN uses distributed texture, contrast, and structural information rather than relying exclusively on the semantic objects shown in an image. This allows many real and generated images to be distinguished successfully, but it also creates characteristic failure modes. Regular high-contrast structures in real images can be interpreted as AI-related artifacts, while natural-looking generated landscapes can be classified as real. The variation between generator families and the performance loss after image modification further show that the learned cues do not generalize equally across all image sources and transformations.